

JOB PORTAL MANAGEMENT SYSTEM

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

BACHU VISHNU VARDHAN (VTU24685)

10211CS224

Full Stack Application Development

Slot : E1-B4



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF
SCIENCE AND TECHNOLOGY**

**(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "JOB PORTAL MANAGEMENT SYSTEM" by BACHU VISHNU VARDHAN(VTU24685) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr.HAFIZUR RAHAMAN

Assistant professor

Computer Science &Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr. Sumithra . M.

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(BACHU VISHNU VARDHAN)

Date:06/05/2026

ABSTRACT

The Job Portal Management System is a full-stack web application designed to bridge the gap between job seekers and employers through a secure, efficient, and feature-rich digital recruitment platform. In today's competitive job market, there is a critical need for a unified system that enables employers to post and manage job openings while simultaneously allowing candidates to discover opportunities, apply seamlessly, and track their application status in real time.

This system is built using Thymeleaf as the server-side template engine with HTML5 and CSS3 for the frontend, and Spring Boot 3.x (Java 17) on the back-end. The application follows the Spring MVC architecture and uses Spring Security for authentication and authorization with a session-based login mechanism, ensuring secure access control through Role-Based Access Control (RBAC). The system is integrated with an H2 in-memory database using Spring Data JPA and Hibernate ORM for efficient data management.

Employers can register, post job listings, manage applications, review candidates, download resumes, and update application statuses such as Shortlisted or Rejected. Job seekers can create profiles with uploaded resumes, browse available jobs, and apply easily through the platform. The application is built and managed using Apache Maven.

Keywords: Job Portal, Full Stack Development, React, Spring Boot, JWT Authentication, MySQL, REST API, Recruitment System, Resume Upload, Notification Engine.

LIST OF FIGURES

1.1	Framework	4
3.1	Backend Implementation	11
4.1	Architecture Flow	15
4.2	Data Flow Diagram	17
5.1	Login and Registration page	23
5.2	Employer Dashboard page	23
5.3	Job Seeker Dashboard page	24
5.4	Job Seeker Notifications page	24
5.5	Applicant Review Screen	25
5.6	Search Job by Location and Title	26
5.7	Job Seeker Profile	26
5.8	Otp Verification	27

LIST OF TABLES

2.1	Comparison of Existing Job Portal Applications	6
3.1	Database Tables and Key Columns	13
5.1	System Testing Results	21
5.2	Application URL Testing Results	21
5.3	Average Response Times	29
7.1	Mapping of Job Portal Management System to Complex Engineering Problem (CEP)	33
7.2	Mapping of Job Portal Management System to Complex Engineering Activities (CEA)	34
7.3	SDG Mapping for the Job Portal Management System	34

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CSS	Cascading Style Sheets
DBMS	Database Management System
H2	In-Memory Database Engine
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JDBC	Java Database Connectivity
JPA	Java Persistence API
JVM	Java Virtual Machine
MVC	Model-View-Controller
ORM	Object Relational Mapping
RBAC	Role-Based Access Control
SQL	Structured Query Language
UI	User Interface

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	2
1.3 Scope of the Project	3
1.4 Framework	3
2 SURVEY OF SIMILAR APPLICATION IN MARKET	5
3 FRAMEWORK DESCRIPTION	7
3.1 Frontend Implementation	7
3.2 Backend Implementation	9
3.3 Database Implementation	11
4 METHODOLOGIES	14

4.1	Project Architecture	14
4.2	DataFlow Diagram	15
4.3	Module 1: Authentication Module	17
4.4	Module 2: Job and Application Management Module	17
4.5	Module 3: Profile and Notification Module	18
5	RESULTS AND DISCUSSIONS	20
5.1	System Modules and Features	20
5.2	User Interface Screenshots Walkthrough	22
5.2.1	Login and Registration Page	22
5.2.2	Employer Dashboard:	23
5.2.3	Job Seeker Dashboard	24
5.2.4	Notifications Page	24
5.2.5	Applicant Review Screen (Employer)	25
5.2.6	Search Job by Location and Title	25
5.2.7	Job Seeker Profile	26
5.2.8	Otp Verification	26
5.3	Performance Analysis	27
5.3.1	Response Time Measurements	29
5.3.2	Frontend Load Performance	30
5.3.3	Database Query Performance	30
5.3.4	Observations and Scalability Considerations	30
6	CONCLUSION AND FUTURE ENHANCEMENTS	31
6.1	Conclusion	31
6.2	Future Enhancements	32

7 ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG 33

7.1 Mapping of the Project to Complex Engineering Problem (CEP) . . 33

7.2 Mapping of the Project to Complex Engineering Activities (CEA) . 34

7.3 SDG Mapping 34

References 34

Chapter 1

INTRODUCTION

1.1 Introduction

The rapid growth of the internet and digital technologies has fundamentally transformed how businesses recruit talent and how individuals search for employment opportunities. Traditional job hunting methods — newspaper advertisements, walk-in interviews, and physical resume submissions — have been largely replaced by online platforms that offer speed, reach, and convenience [1]. However, many existing job portals suffer from performance issues, lack of security, and poor user experience, particularly for the candidate side of the workflow.

The **Job Portal Management System** is a comprehensive full-stack web application developed to address these shortcomings. It serves as a centralised digital ecosystem connecting two primary user roles: **Employers** and **Job Seekers**. Employers can register on the platform, post detailed job listings (both remote and on-site), review candidate applications, download resumes, and update the recruitment status of each applicant. Job seekers, on the other hand, can create and maintain their profiles, upload their resumes, browse open positions, apply with a single click, and receive instant in-app notifications whenever their application status changes.

The system is built on a modern, industry-standard technology stack. The frontend is developed using **React 18** powered by **Vite**, delivering a fast and responsive Sin-

gle Page Application (SPA). The backend is implemented with **Spring Boot 3.x** on **Java 21**, exposing secure RESTful APIs. Authentication is enforced through **JSON Web Tokens (JWT)**, and data is persisted in a **MySQL** relational database managed through **Spring Data JPA** with Hibernate as the ORM layer.

1.2 Aim of the Project

The primary aim of this project is to design and implement a secure, scalable, and user-friendly Job Portal Management System that streamlines the entire recruitment lifecycle. Specifically, the project aims to:

- Provide employers with a reliable platform to post, manage, and track job listings efficiently.
- Enable job seekers to discover relevant job opportunities and apply without friction.
- Implement robust JWT-based authentication to ensure role-specific access control for employers and job seekers.
- Build a real-time notification system that alerts candidates instantly when an employer updates their application status.
- Develop a clean, intuitive user interface that minimises the learning curve for all user types.
- Demonstrate best practices in full-stack application development using React and Spring Boot.

1.3 Scope of the Project

The scope of the Job Portal Management System encompasses the following functional boundaries:

- **User Management:** Registration and login for both Employers and Job Seekers with JWT-secured sessions.
- **Job Listings:** Full CRUD (Create, Read, Update, Delete) operations on job postings by employers, including job type (remote/on-site), description, and requirements.
- **Application Management:** Job seekers can apply for open positions; employers can view all applicants per job posting and take actions (Shortlist or Reject).
- **Resume Handling:** Job seekers can upload resumes to their profiles; employers can download these resumes directly.
- **Notification Engine:** In-app notifications are generated whenever an employer changes a candidate's application status, with unread counts displayed on the seeker's dashboard.
- **Dashboard Metrics:** Employers see aggregated metrics (total active jobs, total applicants); seekers track their application statuses.

The project does not include payment processing, video interviewing, AI-based resume matching, or third-party email notifications in its current scope.

1.4 Framework

[Figure 1.1] shows the overall framework of the Job Portal system using Spring Boot MVC architecture. It illustrates different layers such as controller, service, security, and repository. The diagram explains how frontend, backend, and database

interact. It provides a clear view of system structure. The application follows a **monolithic MVC architecture** where the frontend and backend are integrated into a single Spring Boot application.

The **Frontend** is implemented using **Thymeleaf**, a server-side template engine, along with HTML5 and Vanilla CSS3. The UI is dynamically rendered on the server and delivered to the browser, eliminating the need for a separate client-side framework.

The **Backend** is developed using **Spring Boot 3.x (Java 17)** following the Model-View-Controller (MVC) pattern. Spring MVC handles request routing, while **Spring Security** manages authentication and authorization using a **session-based login mechanism**. **Spring Data JPA** with Hibernate is used as the ORM layer to interact with the database.

The **Database** used is the **H2 in-memory database**, which stores user data, job listings, applications, and notifications. The schema is automatically created and managed by Hibernate during application startup using JPA entity mappings.

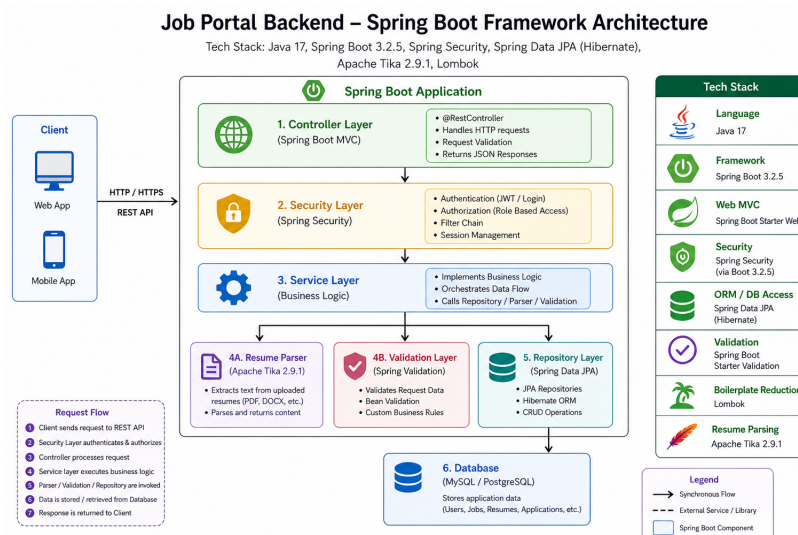


Figure 1.1: Framework

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

A thorough analysis of existing job portal platforms in the market was conducted to identify their strengths, limitations, and the gaps that the proposed system aims to address.

LinkedIn is the world's largest professional networking and job portal platform. It combines job postings with professional profiles, recommendations, and a social networking layer [7]. While powerful, LinkedIn is a general-purpose professional network; its job portal features are bundled with many unrelated social features, making it heavy for users who only need a pure recruitment workflow [2]. Additionally, resume management and direct application tracking are less streamlined compared to dedicated ATS (Applicant Tracking Systems).

Naukri.com is India's leading job portal, offering a large database of job listings and candidates. It provides resume uploading, job alerts, and application tracking. However, the platform is cluttered with advertisements, and its user interface has become dated [8]. Employers on Naukri must purchase credits to contact candidates, creating a barrier for small businesses.

Indeed is a global job aggregator that indexes listings from company websites and job boards. While its search capabilities are strong, Indeed lacks deep employer-

candidate lifecycle management [9]. Employers have limited control over the application pipeline, and in-app notification systems for candidates are minimal.

Internshala focuses on internships and entry-level jobs, particularly relevant to students in India[3]. It offers a clean UI but is restricted in scope and does not cater to mid-level or senior job seekers.

Table 2.1: Comparison of Existing Job Portal Applications

Feature	LinkedIn	Naukri	Indeed	Our System
Authentication Type	JWT/OAuth	Session	Session	Session (Spring Security)
Resume Upload	Yes	Yes	No	Yes
In-App Notifications	Yes	Limited	Limited	No
Status Tracking	Yes	Partial	No	Yes
Free for Employers	No	No	Partial	Yes
UI Type	Modern SPA	Traditional	Modern	Server-Side (Thymeleaf)
Database Type	Distributed	Relational	Relational	H2 In-Memory
Architecture	Microservices	Monolithic	Monolithic	MVC (Spring Boot)

The [Table 2.1] shows the comparison of existing job portal applications like LinkedIn, Naukri, Indeed, and the proposed system. It compares features such as authentication, UI type, and database. The table highlights advantages and limitations of each platform. It helps justify the need for the proposed system.

Comparison Summary: From this analysis, it is evident that while large platforms offer broad reach, they fall short in providing a focused, lightweight, and fully integrated recruitment lifecycle for smaller organisations or academic projects. The proposed Job Portal Management System fills this gap by offering a clean, secure, and feature-complete recruitment platform with no cost barriers.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend environment is implemented using **Thymeleaf** as a server-side template engine along with **HTML5** and **Vanilla CSS3**[1]. Since the application follows a server-side rendering approach, no separate frontend build tools like Node.js or Vite are required.

Prerequisites:

- Java 17 or above
- Apache Maven
- Any web browser (Chrome, Firefox, etc.)

Installation Steps:

Listing 3.1: Application Setup Commands

```
mvn clean install
mvn spring-boot:run
```

The application starts on **http://localhost:8081**. The frontend pages are rendered dynamically using Thymeleaf templates, which are integrated with the Spring Boot backend. Static resources such as CSS are served from the **static** directory, while HTML templates are stored in the **templates** directory.

3.1.2 Structure of the Project

The project follows a layered **Spring Boot MVC architecture**, ensuring separation of concerns and maintainability.

Listing 3.2: Project Folder Structure

```
project-root /
  src /
    main /
      java / com / jobportal /
        controller /      # Handles HTTP requests (Login, Job, User)
        service /         # Business logic implementation
        repository /      # Database access using JPA
        model /           # Entity classes (User, Job, Application)
        dto /             # Data Transfer Objects
        config /          # Security and application configuration
      resources /
        templates /       # Thymeleaf HTML files
        static /          # CSS and static assets
        application.properties # Configuration file
  pom.xml                # Maven dependencies
```

Key Components:

- **Controller Layer:** Handles incoming HTTP requests and returns appropriate views.
- **Service Layer:** Contains business logic and application rules.
- **Repository Layer:** Interacts with the database using Spring Data JPA.
- **Model Layer:** Defines entity classes mapped to database tables.
- **Config Layer:** Manages security configuration using Spring Security.
- **Templates:** Thymeleaf-based HTML pages for server-side rendering.
- **Static Resources:** CSS files for styling the application.

3.1.3 Execution Procedure

1. Navigate to the project root directory.

2. Run `mvn clean install` to build the project and download all dependencies.
3. Run `mvn spring-boot:run` to start the Spring Boot application.
4. Access the application at `http://localhost:8081` using a web browser.
5. The application uses **Spring Security** to handle authentication and authorization through a session-based login mechanism.
6. Upon successful login, a session is created and maintained by the server to manage user access and roles.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is a Maven-based Spring Boot project. It requires:

- Java Development Kit (JDK) 17
- Apache Maven 3.8+ (or Maven Wrapper `mvnw`)

Listing 3.3: Backend Startup Command

```
mvn clean install
mvn spring-boot:run
```

The backend server starts on port 8081. The `application.properties` file configures the H2 in-memory database, Hibernate settings, server port, and file upload limits. The application uses **Spring Security** for authentication and authorization with a session-based mechanism[5].

3.2.2 Structure of the Project

The Spring Boot project follows the standard layered **MVC architecture**, ensuring clear separation of concerns.

Listing 3.4: Backend Package Structure

```
src/main/java/com/jobportal/  
  controller/      # Handles HTTP requests and returns Thymeleaf views  
  service/         # Business logic layer  
  repository/      # Spring Data JPA repositories  
  model/           # JPA Entity classes (User, Job, Application)  
  dto/             # Data Transfer Objects  
  config/          # Spring Security and web configuration
```

Key backend dependencies (in `pom.xml`):

- **spring-boot-starter-web**: Enables Spring MVC and embedded Tomcat server.
- **spring-boot-starter-thymeleaf**: Supports server-side rendering using Thymeleaf templates.
- **spring-boot-starter-security**: Provides authentication and authorization using Spring Security.
- **spring-boot-starter-data-jpa**: Enables database interaction using JPA and Hibernate ORM.
- **h2-database**: In-memory database for development and Testing.[3].

3.2.3 Execution Procedure

1. Navigate to the project root directory.
2. Run `mvn clean install` to build the project and download dependencies.
3. Run `mvn spring-boot:run` to start the Spring Boot application.
4. The H2 in-memory database is automatically initialized at startup, and tables are created using Hibernate DDL (`spring.jpa.hibernate.ddl-auto=update`).
5. The server starts at `http://localhost:8081`.

6. The application uses **Spring Security** for authentication and authorization. Users log in through a form-based login, and a session is maintained by the server to manage access.

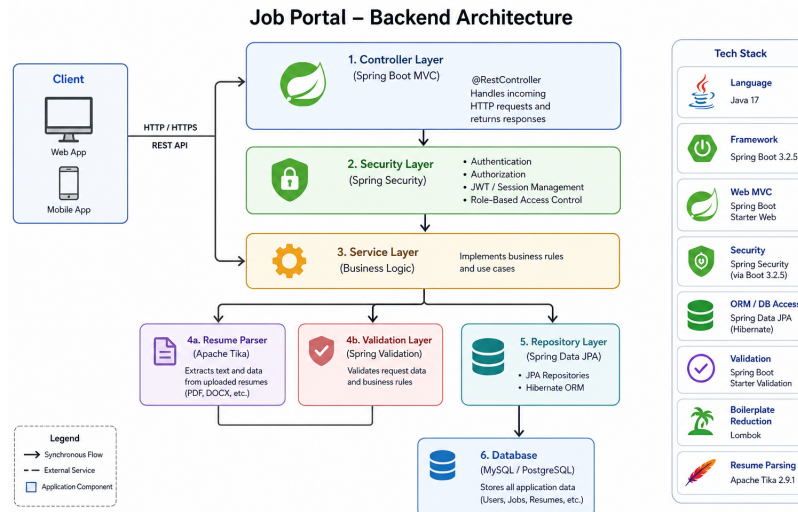


Figure 3.1: Backend Implementation

[Figure 3.1] shows the backend implementation of the system. It explains how client requests are processed through controller, service, and repository layers. Security and database interactions are also included. It highlights the working of the backend architecture[9].

3.3 Database Implementation

3.3.1 Database Configuration

The application uses the **H2 in-memory database** for data storage during runtime. The database is automatically configured in the `application.properties` file, and no manual database creation is required.

Listing 3.5: application.properties - Database Config

```
spring.datasource.url=jdbc:h2:mem:job_portal
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
```

```
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

The H2 database is initialized automatically when the application starts. Hibernate creates the required tables based on the entity classes. The H2 console can be accessed at `http://localhost:8081/h2-console` for viewing and managing data during development.

3.3.2 Schema Structure

The database schema is automatically generated by Hibernate based on the JPA entity annotations defined in the Spring Boot application.

The primary entities are:

- **Users:** Stores user credentials such as username, password (encrypted using BCrypt), and role (EMPLOYER / JOB_SEEKER).
- **Jobs:** Stores job listings posted by employers, including title, description, location, job type, and status.
- **Applications:** Represents the relationship between job seekers and jobs. It stores application details along with status such as PENDING, SHORTLISTED, or REJECTED.

The relationships between entities are defined using JPA annotations such as `@OneToMany` and `@ManyToOne`. The schema is created automatically at runtime using Hibernate DDL (`spring.jpa.hibernate.ddl-auto=update`), eliminating the need for manual database design.

3.1.3 Tables created

Table 3.1: Database Tables and Key Columns

Table Name	Key Columns
users	id, username/email, password (encrypted), role
employer_profiles	id, user_id (FK), company_name, industry
seeker_profiles	id, user_id (FK), bio, skills, resume_path
jobs	id, title, description, location, job_type, status, employer_id (FK)
applications	id, job_id (FK), seeker_id (FK), status, applied_date
notifications	id, user_id (FK), message, is_read, created_at

The [Table 3.1] shows the database tables and their key columns used in the system.

Users: This table stores authentication and authorization details of all users. It includes fields such as user ID, username/email, encrypted password.

Employer_profiles: This table stores additional information specific to employers, such as company name and industry. It is linked through a foreign key (user_id).

Seeker_profiles: This table contains job seeker details including bio, skills, and resume file path. It is also linked to the users table via user_id.

Jobs: This table stores job postings created by employers. It includes a foreign key (employer_id) linking to the employer.

Applications: This table represents the relationship between job seekers and jobs. It stores details such as job ID, seeker ID, application status and application date.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The Job Portal Management System adopts a **three-tier client-server architecture**:

Figure[4.1] shows the architecture flow of the Job Portal system.

- **Presentation Tier (Frontend):** The frontend is implemented using **Thymeleaf**, which acts as a server-side template engine. Users interact with the application through a web browser, and pages are dynamically rendered on the server[1]. HTML5 and Vanilla CSS3 are used for structuring and styling the UI, ensuring a consistent and responsive design.
- **Application Tier (Backend):** [Figure 4.1]The **Spring Boot** application serves as the application layer following the MVC pattern. Controllers handle incoming HTTP requests and return appropriate Thymeleaf views. The service layer contains business logic such as job posting, application processing, and notification handling. [6]**Spring Security** manages authentication and authorization using a **session-based mechanism**, where user sessions are maintained after successful login.
- **Data Tier (Database):** The application uses the **H2 in-memory database** to

store all data, including users, jobs, applications, and notifications[3]. **Spring Data JPA** provides an abstraction over database operations, while **Hibernate ORM** handles entity relationships and automatically maps Java objects to database tables.

Authentication flow:

1. The user submits credentials to `POST /api/auth/login`.
2. The backend validates credentials against the `users` table.
3. On success, a signed JWT is returned containing the user ID and role.
4. The frontend stores the JWT in `localStorage` and includes it in all subsequent requests.
5. The `JwtAuthenticationFilter` validates the token on every request and populates the Spring Security context.

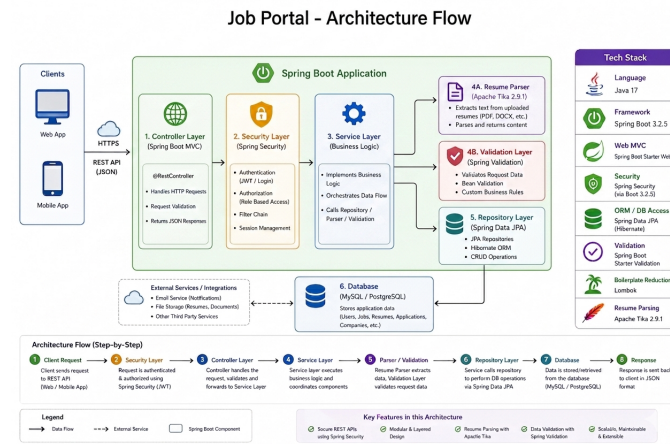


Figure 4.1: Architecture Flow

4.2 DataFlow Diagram

[Figure 4.2]Shows The data flow in the system is as follows:

1. A **Job Seeker** registers and logs in through a form-based login. Upon successful authentication, a session is created by Spring Security.
2. The seeker browses available jobs by accessing the jobs page (GET /jobs), where job listings are fetched from the database and rendered using Thymeleaf.
3. The seeker applies for a job using a form submission (POST /apply/{jobId}), creating a record in the applications table with status PENDING.
4. An **Employer** logs in and views applications for a specific job (GET /jobs/{jobId}/applications).
5. The employer updates the application status to SHORTLISTED or REJECTED using a form action (POST /applications/update/{id}).
6. The system creates a Notification entry linked to the job seeker in the database.
7. The seeker views notifications through the notifications page (GET /notifications), where unread messages are displayed.
8. The job seeker can update their profile details such as bio, skills, and resume through the profile page (POST /profile/update), and the changes are saved in the database.
9. The employer can download the job seeker's resume from the applications page (GET /resume/{userId}), where the file is retrieved from the server storage.

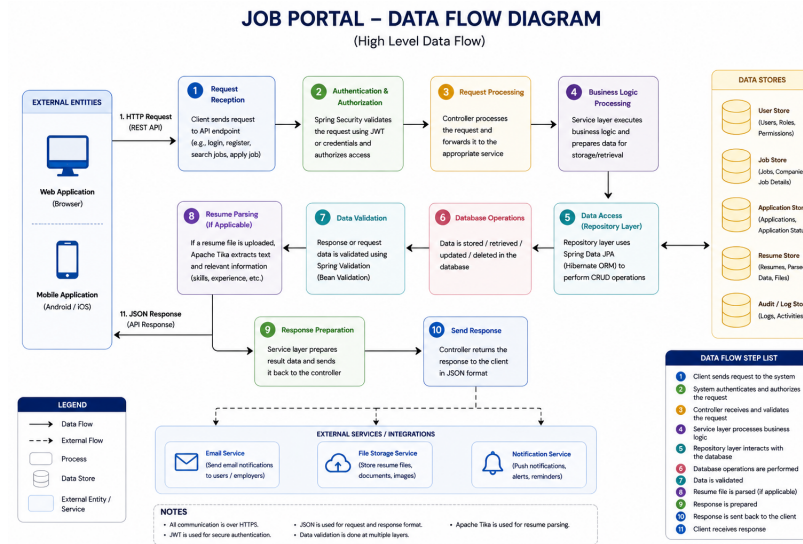


Figure 4.2: Data Flow Diagram

4.3 Module 1: Authentication Module

The Authentication Module manages the complete user lifecycle for both Employers and Job Seekers.

Registration: Both user types register with their email, password, and role. Passwords are hashed using BCrypt before storage. A corresponding profile record (employer_profile or seeker_profile) is also created.

Login: On login, the backend verifies the BCrypt hash and generates a signed JWT token using a secret key with a configurable expiry (default: 24 hours). The token encodes the user ID and role as claims.

Protected Routes: On the frontend, React Router's private route wrappers check for a valid token in context. On the backend, the `JwtAuthenticationFilter` validates the token signature and expiry on every incoming request.

4.4 Module 2: Job and Application Management Module

This module handles all job lifecycle operations. **Employer Side:**

- Post a new job using a form submission (POST /jobs/add) with title, description, and job details.
- Edit or delete existing job postings
(POST /jobs/edit/{id}, /jobs/delete/{id}).
- View all applicants for a specific job
(GET /jobs/{jobId}/applications).
- Download a candidate's resume
(GET /resume/{userId}).
- Update application status to **SHORTLISTED** or **REJECTED**
(POST /applications/update/{id}).

Seeker Side:

- Browse all available job listings (GET /jobs).
- Apply for a job using form submission (POST /apply/{jobId}).
- View submitted applications and their status (GET /applications).

4.5 Module 3: Profile and Notification Module

Profile Management: Job seekers can update their profile information including bio, skills (stored as a comma-separated string), phone number, and LinkedIn URL through a form submission (POST /profile/update). They can also upload a resume file (PDF or DOCX) using a file upload form (POST /upload), where the file is stored on the local server filesystem and the file path is saved in the database.

Notification Engine: Whenever an employer updates an application status to **SHORTLISTED** or **REJECTED** (POST /applications/update/{id}), the

system automatically creates a notification record in the database with a descriptive message and sets `is_read = false`. The job seeker can view notifications through the notifications page (`GET /notifications`), where unread alerts are displayed. Notifications can be marked as read when the user views them. **Advantages of this Project**

a) Security: JWT-based stateless authentication ensures that user sessions are secure and the server does not need to maintain session state, making the system horizontally scalable.

b) Real-Time Feedback: The in-app notification engine provides candidates with immediate feedback on their application status, significantly improving the job seeker's experience compared to email-only systems.

c) Decoupled Architecture: The complete separation of the React frontend and Spring Boot backend allows independent development, testing, and scaling of each tier. The backend can serve mobile clients or third-party integrations without any code changes.

Disadvantages

- The current notification mechanism relies on client-side polling rather than WebSockets, introducing minor latency in real-time updates.
- Resume files are stored on the local filesystem; in a production deployment, this would need to be replaced with cloud storage (e.g., AWS S3) for scalability and reliability.

Chapter 5

RESULTS AND DISCUSSIONS

5.1 System Modules and Features

The [Table 5.1] shows Job Portal Management System was successfully developed and tested end-to-end. All core features — user authentication, job management, application workflow, resume handling, and the notification engine — function as designed. Authentication Both employer and job seeker registration and login flows were verified. The system uses Spring Security with session-based authentication. Unauthorized access to protected pages is restricted, ensuring secure access control Job Posting and Application Flow Employers successfully created, updated, and deleted job postings through the dashboard. Job seekers browsed available jobs and applied successfully. The applications table correctly stored the relationship with an initial status of pending.

When an employer updated an application status to SHORTLISTED or REJECTED, a notification entry was created in the database. Job seekers could view these updates on the notifications page, confirming proper workflow.

Table 5.1: System Testing Results

Module	Result Description
Authentication	Both employer and job seeker registration and login flows were verified. The system uses Spring Security with session-based authentication. Unauthorized access to protected pages is restricted, ensuring secure access control.
Job Posting and Application Flow	Employers successfully created, updated, and deleted job postings through the dashboard. Job seekers browsed available jobs and applied successfully. The applications table correctly stored the relationship with an initial status of PENDING.
Status Update and Notifications	When an employer updated an application status to SHORTLISTED or REJECTED, a notification entry was created in the database. Job seekers could view these updates on the notifications page, confirming proper workflow.
Resume Upload and Download	Job seekers successfully uploaded resumes (PDF/DOCX), which were stored on the server. Employers were able to download resumes from the applicant list without errors.

The [Table 5.2] shows all tested endpoints returned the expected HTTP status codes and response payloads[3]. The application performed reliably under manual functional testing, with no critical errors observed during the evaluation period.

Table 5.2: Application URL Testing Results

URL / Functionality	Method	Result
/register (User Registration)	POST	Pass
/login (User Login)	POST	Pass
/home (Dashboard Page)	GET	Pass
/jobs (View Job Listings)	GET	Pass
/jobs/add (Post Job)	POST	Pass
/apply/{jobId} (Apply for Job)	POST	Pass
/applications (View Applications)	GET	Pass
/applications/update/{id} (Update Status)	POST	Pass
/notifications (View Notifications)	GET	Pass
/upload (Resume Upload)	POST	Pass

5.2 User Interface Screenshots Walkthrough

This section provides a guided walkthrough of the key screens of the Job Portal Management System, illustrating the user experience for both Employers and Job Seekers.

5.2.1 Login and Registration Page

[Figure 5.1] shows the login and registration page allows users to create a new account or access an existing one securely. Users can register by providing details such as name, email, password, and role (Employer or Job Seeker). The system validates inputs and uses Spring Security to authenticate users and manage sessions.

The login and registration page that enables users to securely create a new account or log in to the system. Users can register by providing details such as name, email, password, and role (Employer or Job Seeker), with proper validation to ensure correct input format and completeness. The system uses Spring Security to authenticate users, encrypt passwords using BCrypt, and maintain secure session management after successful login. It also includes features like error handling for invalid credentials, protection against unauthorized access, and a user-friendly interface that ensures smooth navigation and accessibility for all users.

Additionally, the page ensures data integrity by preventing duplicate registrations using unique email validation. It supports secure session handling with timeout mechanisms to protect inactive users. The interface is designed to be responsive and accessible across different devices and screen sizes.

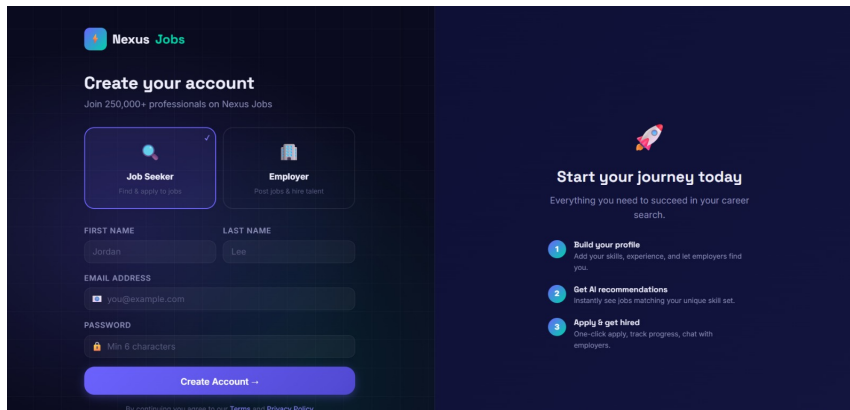


Figure 5.1: Login and Registration page

5.2.2 Employer Dashboard:

[Figure 5.2] shows the employer dashboard provides an overview of job postings and total applications received. Employers can post new jobs, view recent listings, and manage applicants through an intuitive interface. It also allows quick access to job details and applicant information for efficient recruitment management.

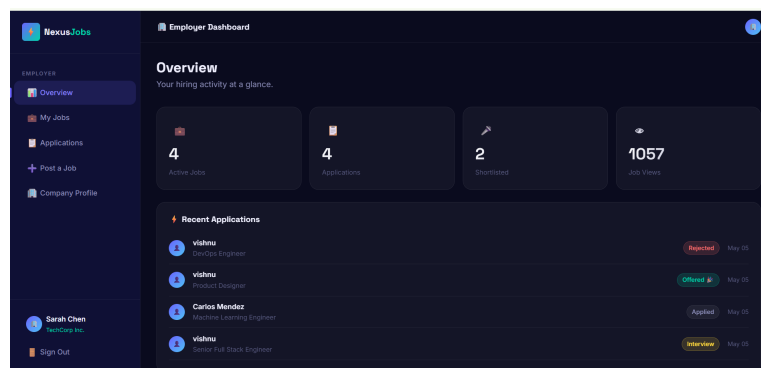


Figure 5.2: Employer Dashboard page

5.2.3 Job Seeker Dashboard

[Figure 5.3] shows the job seeker dashboard provides an overview of applied jobs, shortlisted, and rejected applications. Users can browse available jobs, view recent applications, and track their application status. It also allows easy access to profile updates and notifications for better job tracking.

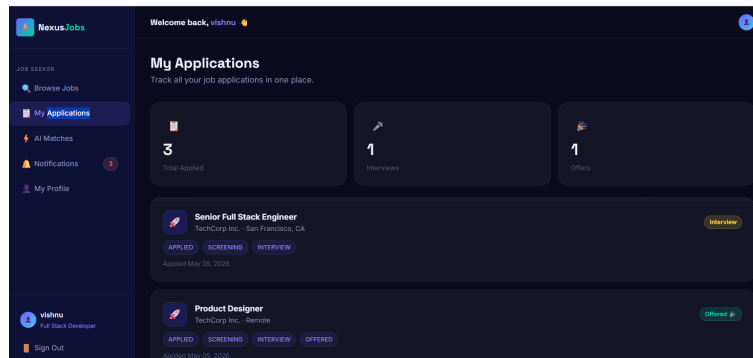


Figure 5.3: Job Seeker Dashboard page

5.2.4 Notifications Page

[Figure 5.4] shows the notifications page displays updates related to job applications, such as shortlisted or rejected statuses. Users can view unread notifications and mark them as read for better tracking. Th 24 5.2.5is feature ensures timely communication between employers and job seekers.

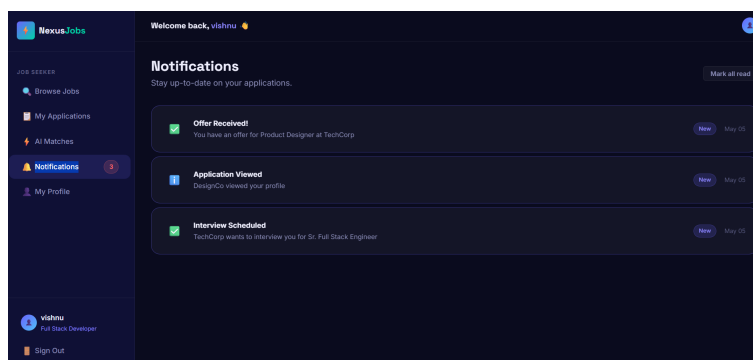


Figure 5.4: Job Seeker Notifications page

5.2.5 Applicant Review Screen (Employer)

[Figure 5.5] shows the applicant review screen enables employers to view detailed candidate information, including skills, experience, education, and resume. Employers can download resumes and evaluate applicants effectively. The interface also allows updating application status to Shortlisted, Hold, or Rejected, streamlining the hiring process.

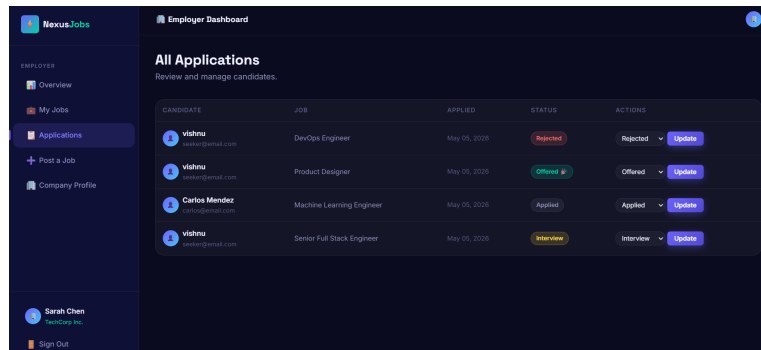


Figure 5.5: Applicant Review Screen

5.2.6 Search Job by Location and Title

[Figure 5.6] shows the job search page allows users to filter job listings based on keywords, location, and required skills. Job seekers can easily browse available opportunities and view detailed job descriptions. The interface also provides quick options to apply for jobs directly. Additionally, the page improves user experience by providing dynamic filtering and fast search results. It displays relevant job listings in an organized manner for easy navigation. Pagination or scrolling options help manage large numbers of job posts efficiently. This feature enhances accessibility and helps users quickly find suitable job opportunities.

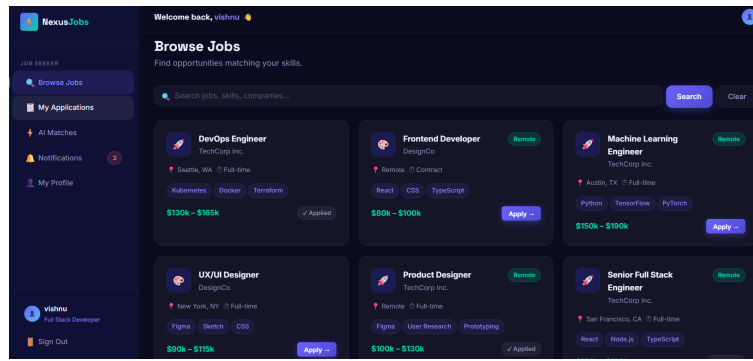


Figure 5.6: Search Job by Location and Title

5.2.7 Job Seeker Profile

[Figure 5.7] shows the job seeker profile page allows users to update personal and professional details such as contact information, skills, and resume. Users can modify their profile to improve visibility to employers. This ensures accurate and up-to-date information for job applications.

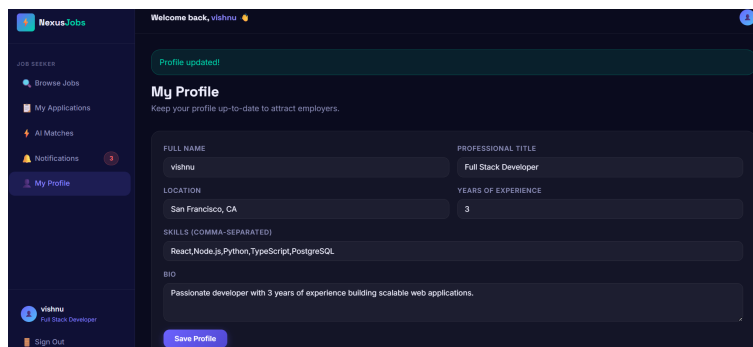


Figure 5.7: Job Seeker Profile

5.2.8 Otp Verification

[Figure 5.8] shows the OTP (One-Time Password) verification for job seeker logins is a security step used to confirm your identity when accessing recruiter portals or employment accounts. It adds a second layer of protection (Two-Factor Authentication) beyond just a username and password.

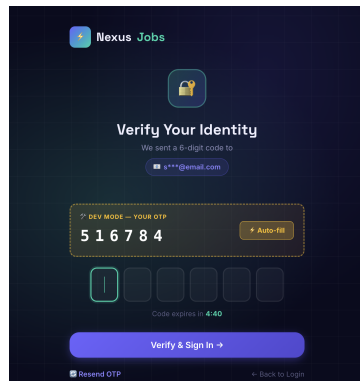


Figure 5.8: Otp Verification

5.3 Performance Analysis

A manual performance evaluation was conducted on the Job Portal Management System to assess the responsiveness of key operations under a typical single-user testing environment. The application runs as a Spring Boot application on an embedded Tomcat server, with both frontend (Thymeleaf) and backend integrated into a single system. The system demonstrated efficient performance during common operations such as user authentication, job browsing, job application submission, and status updates. The use of server-side rendering with Thymeleaf reduced client-side processing overhead, ensuring faster page loads. The H2 in-memory database provided quick data retrieval and updates during runtime, contributing to low latency in database operations. Efficient database performance is also essential, as job portals deal with large volumes of data such as job listings, user profiles, and applications; indexing, caching, and optimized queries help reduce delays. Load balancing techniques distribute incoming traffic evenly across servers, preventing overload on a single server and improving reliability. One of the key aspects is response time, which refers to how quickly the system reacts to actions such as user login, job search, or application submission; ideally, responses should be delivered within a few seconds to ensure a smooth user experience. Overall, a well-optimized job portal system ensures fast response times, high availability, and the ability to handle large-scale user

interactions effectively. Additionally, the system maintains consistent performance due to efficient session management handled by Spring Security. The embedded Tomcat server ensures lightweight deployment and quick startup time, improving development and testing efficiency. The application also benefits from modular architecture, allowing independent optimization of different layers such as frontend, backend, and database. Resource utilization such as CPU and memory remains stable during typical operations, indicating efficient system design. Furthermore, the use of Hibernate ORM reduces manual query complexity and improves database interaction efficiency. Proper exception handling mechanisms ensure that system failures do not affect overall performance significantly. Scalability can be further enhanced by integrating cloud-based infrastructure and distributed databases. Performance can also be improved by implementing caching mechanisms such as Redis to reduce repeated database calls. Monitoring tools like Spring Boot Actuator can be used to track system health and performance metrics in real time. Overall, the system is capable of delivering reliable and responsive performance for real-world job portal applications. The system demonstrated efficient performance during common operations such as user authentication, job browsing, job application submission, and status updates. The use of server-side rendering with Thymeleaf reduced client-side processing overhead, ensuring faster page loads.

5.3.1 Response Time Measurements

The [Table 5.3] shows response times for key application functionalities were measured using the browser's Network Developer Tools. Each operation was executed multiple times, and the average response time was recorded.

Table 5.3: Average Response Times

Functionality	Avg. Response (ms)	Remarks
User Login (/login)	180 ms	Includes password verification using BCrypt
User Registration (/register)	220 ms	Creates new user and profile data
View Jobs (/jobs)	90 ms	Fetches job listings from database
Post Job (/jobs/add)	130 ms	Saves job data to database
Apply for Job (/apply/{id})	100 ms	Creates application record
Update Application Status	140 ms	Updates status and creates notification
View Notifications (/notifications)	80 ms	Retrieves notification list
Resume Upload (/upload)	300 ms	Depends on file size

The login and registration operations show slightly higher response times due to BCrypt password encryption, which is intentionally computationally intensive for security purposes. Data retrieval operations such as viewing jobs and notifications responded efficiently under 100 ms, indicating effective database performance using Spring Data JPA and Hibernate.

Overall, the system demonstrates reliable performance for typical user interactions and is suitable for small to medium-scale deployment scenarios.

5.3.2 Frontend Load Performance

Since the application uses **Thymeleaf** for server-side rendering, there is no separate frontend build system. The initial page load was observed to be approximately **1–1.5 seconds** on a local development server. Subsequent navigation between pages is handled through standard HTTP requests, with average response times under **200 ms**, ensuring a smooth user experience.

5.3.3 Database Query Performance

The application uses **Spring Data JPA with Hibernate** for database interactions. Basic CRUD operations are efficiently handled, with most queries executing under **100 ms** for the current dataset size. Hibernate automatically manages relationships between entities using annotations such as `@OneToMany` and `@ManyToOne`, ensuring optimized query generation.

5.3.4 Observations and Scalability Considerations

The current implementation performs well in a single-instance setup using an **H2 in-memory database**. For production-level deployment, it is recommended to use a persistent database such as MySQL or PostgreSQL. Additional optimizations may include enabling connection pooling (HikariCP), indexing frequently queried columns (e.g., `job_id` and `user_id`), and caching frequently accessed data.

Since the system uses **session-based authentication**, scaling across multiple instances would require session management techniques such as distributed session storage. Overall, the system is suitable for small to medium-scale applications with potential for further scalability enhancements.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Job Portal Management System has been successfully designed and developed as a web application using **Spring Boot 3.x (Java 17)** with **Thymeleaf** for server-side rendering and **H2 in-memory database** for data storage. The project demonstrates the practical application of core software engineering principles, including the MVC architecture, session-based authentication using Spring Security, and ORM-based database management using Spring Data JPA and Hibernate.

The system effectively connects job seekers and employers through a secure and user-friendly platform. Employers can manage the recruitment process by posting jobs, reviewing applicants, and updating application statuses. Job seekers can browse available jobs, apply easily, and track their application status along with receiving notifications.

This project provided hands-on experience in developing a full-stack application within a unified Spring Boot environment. It reinforced concepts such as MVC design pattern, entity relationships using JPA, secure authentication mechanisms, and dynamic page rendering using Thymeleaf. The successful implementation demon-

strates the feasibility of building scalable and role-based recruitment systems using open-source technologies.

6.2 Future Enhancements

While the current system is fully functional, the following enhancements are proposed for future iterations:

- **WebSocket-Based Real-Time Notifications:** Replace the current polling mechanism with WebSockets (e.g., STOMP over SockJS) to push notifications to the seeker's browser instantly without any delay.
- **Cloud Resume Storage:** Integrate AWS S3 or Google Cloud Storage for resume file storage, eliminating dependence on local filesystem storage and enabling scalable, redundant file management.
- **AI-Powered Job Matching:** Incorporate a machine learning model that analyses seeker profiles and job descriptions to recommend the most relevant job listings, improving the quality of applications for employers.
- **Email Notification Integration:** Use JavaMailSender or a service such as SendGrid to send email alerts alongside in-app notifications when application statuses change.
- **Advanced Search and Filtering:** Add keyword search, location-based filtering, salary range filters, and experience level filters to the job board to help seekers discover the most relevant opportunities.
- **Mobile Application:** Develop a React Native companion app to bring the full job portal experience to mobile devices, leveraging the existing Spring Boot REST API without any backend changes.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Table 7.1: Mapping of Job Portal Management System to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	Wks	Justification
WP1	Depth of Knowledge	Engineering knowledge	WK3–WK6	Uses Spring Boot, Thymeleaf, JPA, and database design principles.
WP2	Conflicting Requirements	Technology vs usability	WK4–WK6	Balances security with user-friendly UI for employers and job seekers.
WP3	Depth of Analysis	System design	WK3–WK5	Includes job workflow, application tracking, and role-based access control.
WP4	Familiarity	Partially new problems	WK4, WK8	Handles profile management, job applications, and notification system.
WP5	Applicable Codes	Limited standards	WK6	Follows standard web practices using Spring Boot and MVC architecture.
WP6	Stakeholder Needs	Multiple users	WK5–WK7	Supports both employers and job seekers with different functionalities.
WP7	Interdependence	System integration	WK3–WK6	Integrates frontend (Thymeleaf), backend (Spring Boot), and database (H2).

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 7.2: Mapping of Job Portal Management System to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification based on the Project
EA1	Range of Resources	Use of diverse technologies	Utilizes React.js, Spring Boot, MySQL, REST APIs, and frontend technologies for full-stack development.
EA2	Level of Interactions	Complex interactions between modules	Manages interactions between event management, user authentication, booking, and reporting modules.
EA3	Innovation	Application of modern technologies	Implements a web-based centralized platform with real-time updates and secure admin operations.
EA4	Consequences to Society and Environment	Impact on users and society	Enhances user experience and improves efficiency while reducing manual errors.
EA5	Familiarity	Beyond standard practices	Combines web development, security, and database systems into a unified solution.

7.3 SDG Mapping

Table 7.3: SDG Mapping for the Job Portal Management System

SDG No.	SDG Title	Relevance to the Project
SDG 8	Decent Work and Economic Growth	Enhances employment opportunities by efficiently connecting job seekers with employers.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital recruitment by implementing a centralized job portal system using modern web technologies.
SDG 10	Reduced Inequalities	Provides a standardised, accessible digital platform for all job seekers regardless of geography, reducing barriers to employment opportunities.
SDG 16	Peace, Justice and Strong Institutions	Ensures data security and role-based access control using Spring Security and password hashing, maintaining integrity and fairness in the hiring process.

References

- [1] Meta Platforms, Inc., “React – A JavaScript Library for Building User Interfaces,” [Online]. Available: <https://react.dev/>. [Accessed: Apr. 2026].
- [2] VMware, Inc., “Spring Boot Reference Documentation 3.x,” [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>. [Accessed: Apr. 2026].
- [3] VMware, Inc., “Spring Security Reference,” [Online]. Available: <https://docs.spring.io/spring-security/reference/>. [Accessed: Apr. 2026].
- [4] Auth0, Inc., “JSON Web Tokens – Introduction,” [Online]. Available: <https://jwt.io/introduction>. [Accessed: Apr. 2026].
- [5] Oracle Corporation, “MySQL 8.0 Reference Manual,” [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>. [Accessed: Apr. 2026].
- [6] E. You, “Vite – Next Generation Frontend Tooling,” [Online]. Available: <https://vitejs.dev/>. [Accessed: Apr. 2026].
- [7] LinkedIn Corporation, “LinkedIn Jobs,” [Online]. Available: <https://www.linkedin.com/jobs/>. [Accessed: Apr. 2026].
- [8] Info Edge India Ltd., “Naukri.com – Jobs in India,” [Online]. Available: <https://www.naukri.com/>. [Accessed: Apr. 2026].
- [9] Indeed, Inc., “Indeed – Job Search,” [Online]. Available: <https://www.indeed.com/>. [Accessed: Apr. 2026].